

Cenário Vamos modelar um sistema de pedidos. O **Pedido (Order)** é uma entidade porque:

- Ele tem um `Id` único.
- É um conceito do domínio que pode mudar ao longo do tempo (adicionar itens, alterar status).

Implementação da Entidade

```
public class Order
{
    public int Id { get; private set; }
    public string CustomerName { get; private set; }
    public List<OrderItem> Items { get; private set; } = new
List<OrderItem>();
    public DateTime CreatedAt { get; private set; }

    public Order(int id, string customerName)
    {
        if (string.IsNullOrEmpty(customerName))
        {
            throw new ArgumentException("Customer name cannot be null or
empty.");
        }
        Id = id;
        CustomerName = customerName;
        CreatedAt = DateTime.UtcNow;
    }

    public void AddItem(OrderItem item)
    {
        if (item == null)
        {
            throw new ArgumentNullException(nameof(item));
        }

        Items.Add(item);
    }

    public void RemoveItem(OrderItem item)
    {
        if (item == null)
        {
            throw new ArgumentNullException(nameof(item));
        }

        Items.Remove(item);
    }
}
```

Exemplo de Item do Pedido (Entidade Relacionada)

```
public class OrderItem
{
    public int Id { get; private set; }
    public string ProductName { get; private set; }
    public int Quantity { get; private set; }
    public decimal Price { get; private set; }

    public OrderItem(int id, string productName, int quantity, decimal
price)
    {
        if (string.IsNullOrEmpty(productName))
        {
            throw new ArgumentException("Product name cannot be null or
empty.");
        }

        if (quantity <= 0)
        {
            throw new ArgumentException("Quantity must be greater than
zero.");
        }

        if (price <= 0)
        {
            throw new ArgumentException("Price must be greater than
zero.");
        }

        Id = id;
        ProductName = productName;
        Quantity = quantity;
        Price = price;
    }
}
```

Exemplo de Uso

```
// Criando o pedido
var order = new Order(1, "John Doe");

// Adicionando itens ao pedido
var item1 = new OrderItem(1, "Laptop", 1, 1500.00m);
var item2 = new OrderItem(2, "Mouse", 2, 50.00m);

order.AddItem(item1);
order.AddItem(item2);

// Exibindo detalhes do pedido
Console.WriteLine($"Order ID: {order.Id}");
Console.WriteLine($"Customer: {order.CustomerName}");
Console.WriteLine("Items:");
foreach (var item in order.Items)
{
    Console.WriteLine($" - {item.ProductName}, Quantity: {item.Quantity},
Price: {item.Price:C}");
}

// Removendo um item
order.RemoveItem(item2);

Console.WriteLine("Updated Items:");
foreach (var item in order.Items)
{
    Console.WriteLine($" - {item.ProductName}, Quantity: {item.Quantity},
Price: {item.Price:C}");
}
```

Diferença entre Entidades e Value Objects

Aspecto	Entity	Value Object
Identidade	Identidade única (<code>Id</code>).	Não possui identidade única.
Comparação	Comparada por referência ou <code>Id</code> .	Comparada por valores.
Persistência	Geralmente armazenada no banco.	Geralmente parte de uma entidade.
Mutabilidade	Pode ser mutável.	É imutável.

Quando usar Entidades?

- Para representar objetos do domínio que precisam de uma identidade única e persistente (ex.: Pedido, Cliente, Produto).
- Quando o conceito modelado pode mudar ao longo do tempo.